An Experimental Comparison of SAT Encodings, Constraint Programming, and Mixed-Integer Programming for the N-Queens Problem

Nguyen Van Chien[a]

[a] *Faculty of Information Technology, VNU University of Engineering and Technology , Hanoi, Vietnam*

ARTICLE INFO

Keywords:
N-Queens problem
SAT encoding
At-most-one constraints
Constraint programming
Mixed-integer programming
Experimental evaluation

ABSTRACT

This study empirically compares SAT encodings, constraint programming, and mixed-integer programming for finding one feasible solution to the N-Queens problem. Five manually implemented at-most-one encodings, namely Pairwise, Binary, Commander, recursive Product, and Sequential Counter, are evaluated using Glucose3 and CaDiCaL 1.9.5. Four additional configurations use OR-Tools CP-SAT, IBM CP Optimizer, IBM CPLEX MIP, and Gurobi MIP. The experiment covers 18 board sizes ranging from $N = 4$ to $N = 1331$, with three repetitions per configuration and size, yielding 756 runs. Each run has an external wall-time limit of 100 seconds and a sampled process-tree RSS limit of 8 GiB. All returned placements are independently validated. Overall, 612 runs produce valid solutions, 101 reach the time limit, 42 encounter memory limits or allocation failures, and one terminates abnormally. IBM CP Optimizer and Commander encoding with Glucose3 each solve all 54 planned runs. At $N = 1331$, their median total wall times are 1.09 and 65.64 seconds, respectively. Product encoding achieves 21 successful runs with Glucose3 and 48 with CaDiCaL, highlighting the dependence of observed performance on both the encoding and the solver. The findings apply to the tested implementations and resource budgets. Interpretation is limited by three repetitions with default solver seeds, changes to the paging-file configuration and process cleanup after the first 239 runs, and runtime and memory summaries restricted to successful runs.

1. Introduction

The N-Queens problem requires placing N queens on an $N \times N$ chessboard such that no two queens share a row, column, or diagonal. Its compact definition makes it a useful case study for examining how different mathematical representations affect the behavior of general-purpose solvers. This report considers the ordinary feasibility problem: finding one valid placement on an initially empty board. It does not enumerate all solutions or study N-Queens Completion, where some queens are fixed in advance. This distinction matters because the ordinary problem admits constructive solutions, whereas N-Queens Completion is NP-complete [1]. The purpose of this study is therefore to compare solver implementations and modeling choices, rather than to establish the intrinsic difficulty of constructing an N-Queens placement.

The same feasibility conditions can be represented using Boolean satisfiability (SAT), constraint programming (CP), or mixed-integer programming (MIP). In the SAT formulation used here, Boolean variables indicate whether individual board cells contain queens. Exactly-one constraints are imposed on rows and columns, while at-most-one constraints prevent diagonal conflicts. The CP formulation instead uses one integer variable per row and three global distinctness constraints. The MIP formulation uses binary cell variables with linear equality and inequality constraints. Although these models describe the same valid placements, their representations and processing requirements differ.

Within the SAT formulation, the encoding of at-most-one constraints provides a further modeling choice. This study implements five encodings: Pairwise, Binary, Commander, recursive Product, and Sequential Counter. All five use the same board-variable mapping and at-least-one clauses, allowing the comparison to focus on their different at-most-one constructions. The encodings are implemented manually, while PySAT provides the interface to Glucose3 and CaDiCaL 1.9.5 [2]. The comparison also includes OR-Tools CP-SAT, IBM CP Optimizer, IBM CPLEX MIP, and Gurobi MIP.

The experimental evaluation comprises 14 configurations across 18 board sizes, with three repetitions for each configuration and size. This produces 756 planned runs under a common external wall-time budget of 100 seconds and a sampled process-tree RSS limit of 8 GiB. Every returned placement is independently checked for validity. The

recorded outputs distinguish model construction, solver-call time, total process time, and observed peak memory. Timeouts, memory failures, and abnormal termination are retained alongside successful runs.

The study examines how encoding choice and solver choice relate to solution success, runtime, memory consumption, and CNF size. Its contribution is a documented implementation and an auditable experimental comparison of these configurations. Conclusions are restricted to the tested settings, with explicit consideration of the limited repetitions and changes to the execution environment during the experiment.

The remainder of this report is organized as follows. Section 2 introduces the problem formulation and the CP and MIP models. Section 3 describes the SAT encodings and implementation. Section 4 presents the experimental protocol, results, and limitations. Section 5 concludes the report.

2. Background

2.1. Problem definition and notation

Let $N \geq 1$ denote the board size, and let $\mathcal{I} = \{0, 1, \ldots, N-1\}$ be the index set for both rows and columns. A placement is represented by a vector $q = (q_0, q_1, \ldots, q_{N-1})$, where q_i denotes the column occupied by the queen in row i.

A valid placement satisfies

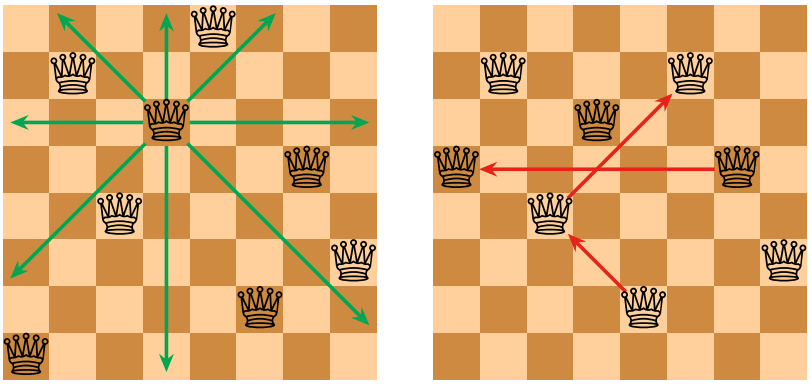
$$q_i \in \mathcal{I}, \qquad \forall i \in \mathcal{I}, \tag{1}$$

$$q_i \neq q_j, \qquad \forall\, 0 \leq i < j < N, \tag{2}$$

$$|q_i - q_j| \neq |i - j|, \qquad \forall\, 0 \leq i < j < N. \tag{3}$$

The vector representation assigns exactly one queen to each row. Equation (2) prevents two queens from sharing a column, while Equation (3) prevents diagonal attacks. Equivalently, diagonal validity requires the values $q_i + i$ to be pairwise distinct and the values $q_i - i$ to be pairwise distinct.

Figure 1 illustrates valid and invalid placements on an 8×8 board. In the valid placement, no two queens share a row, column, or diagonal. Green arrows indicate the eight attack directions of one selected queen. In the invalid placement, red arrows highlight examples of conflicts between queens.



(a) Valid placement (b) Invalid placement

Figure 1: Valid and invalid placements of eight queens. Green arrows show the attack directions of a selected queen in a valid solution; red arrows highlight selected conflicts in an invalid placement.

The ordinary problem is feasible for $N = 1$ and every $N \geq 4$, whereas $N = 2$ and $N = 3$ are infeasible [1]. These small cases are useful for checking both satisfiable and unsatisfiable outcomes.

The computational task in this report is to return one valid placement or establish infeasibility. Every placement returned by a solver is checked independently using its length, column domains, and the distinctness of columns and both diagonal indices. Reaching a time or memory limit is recorded as a resource-limited outcome and does not establish infeasibility.

2.2. Constraint programming formulation

The constraint programming formulation uses N integer variables q_i, each with domain $\mathcal{I}$. The non-attacking conditions are expressed through three global constraints:

$$\begin{aligned} &\mathrm{AllDifferent}\big((q_i)_{i\in\mathcal{I}}\big),\\ &\mathrm{AllDifferent}\big((q_i+i)_{i\in\mathcal{I}}\big),\\ &\mathrm{AllDifferent}\big((q_i-i)_{i\in\mathcal{I}}\big). \end{aligned} \tag{4}$$

Here, AllDifferent requires all expressions in its argument to take different values. The first constraint enforces column uniqueness. The second and third enforce uniqueness along the two diagonal directions. Row uniqueness is already implicit in the variable representation.

Both IBM CP Optimizer and OR-Tools CP-SAT receive this formulation in the implementation. Each is configured to use one worker. No objective function or additional manual symmetry-breaking constraint is introduced.

At the modeling interface, this formulation contains N integer variables and three global constraints. These counts describe the submitted model; preprocessing and internal representations may differ between the two backends. Consequently, the interface-level counts alone do not determine runtime or memory consumption.

2.3. Mixed-integer programming formulation

The MIP formulation introduces a binary variable for each cell:

$$x_{ij} = \begin{cases} 1, & \text{if a queen occupies cell } (i,j),\\ 0, & \text{otherwise,} \end{cases} \qquad i,j \in \mathcal{I}. \tag{5}$$

For the two diagonal directions, define

$$D_d^- = \{(i,j) \in \mathcal{I}^2 : i-j = d\}, \qquad 1-N \le d \le N-1, \tag{6}$$

$$D_s^+ = \{(i,j) \in \mathcal{I}^2 : i+j = s\}, \qquad 0 \le s \le 2N-2. \tag{7}$$

The feasibility constraints are

$$\sum_{j\in\mathcal{I}} x_{ij} = 1, \qquad \forall i \in \mathcal{I}, \tag{8}$$

$$\sum_{i\in\mathcal{I}} x_{ij} = 1, \qquad \forall j \in \mathcal{I}, \tag{9}$$

$$\sum_{(i,j)\in D_d^-} x_{ij} \le 1, \qquad \forall d \text{ with } |D_d^-| \ge 2, \tag{10}$$

$$\sum_{(i,j)\in D_s^+} x_{ij} \le 1, \qquad \forall s \text{ with } |D_s^+| \ge 2, \tag{11}$$

$$x_{ij} \in \{0,1\}, \qquad \forall i,j \in \mathcal{I}. \tag{12}$$

Equations (8) and (9) place exactly one queen in every row and column. Equations (10) and (11) allow at most one queen on each diagonal containing at least two cells. Singleton-diagonal inequalities are omitted because they are already implied by the binary variable domains.

For $N \ge 2$, each diagonal direction contains $2N-3$ diagonals of length at least two. The implemented formulation therefore has N^2 binary variables and

$$2N + 2(2N-3) = 6N-6 \tag{13}$$

linear constraints before presolve, excluding variable bounds. For the special case $N = 1$, the implementation creates two row/column equalities and no diagonal inequalities.

The same formulation is implemented with IBM CPLEX MIP and Gurobi MIP. Both wrappers specify the constant objective $\min 0$, so every feasible placement has the same objective value. Although MIP solver interfaces are used, the submitted model is a pure binary integer linear program with no continuous decision variables. Both solvers use one thread, and no additional manual symmetry-breaking constraints are imposed.

A returned binary solution is decoded into the common vector representation q by identifying the selected column in each row. The resulting placement is then checked using the same independent validator as the SAT and CP solutions.

3. Methodology

This section describes the common SAT formulation, the five implemented at-most-one encodings, their variable and clause counts, and the procedures used to check correctness. All encodings share the same board representation and constraint groups. Only the translation of at-most-one constraints changes between configurations.

3.1. Common SAT formulation

For each cell $(i, j) \in \mathcal{I}^2$, introduce a Boolean variable x_{ij} that is true precisely when the cell contains a queen. The implementation assigns the integer identifier

$$\mathrm{id}(x_{ij}) = 1 + iN + j. \tag{14}$$

Thus, the board variables occupy identifiers $1, \dots, N^2$. Auxiliary variables are allocated consecutively above N^2, with disjoint identifiers for different constraint groups.

For a group $Y = (y_1, \dots, y_m)$, define

$$\mathrm{ALO}(Y) \equiv \bigvee_{i=1}^{m} y_i, \tag{15}$$

$$\mathrm{AMO}(Y) \equiv \sum_{i=1}^{m} y_i \le 1, \tag{16}$$

where Boolean values in the sum are interpreted as zero or one. An exactly-one constraint is the conjunction of ALO and AMO.

Let $\mathcal{R}$ and $\mathcal{C}$ denote the collections of row and column groups, respectively. Let $\mathcal{D}$ contain the groups for both diagonal directions, retaining only diagonals with at least two cells. For encoding E, the generated formula is

$$\begin{aligned} F_E(N) = & \bigwedge_{G \in \mathcal{R} \cup \mathcal{C}} \mathrm{ALO}(G) \\ & \wedge \bigwedge_{G \in \mathcal{R} \cup \mathcal{C} \cup \mathcal{D}} \mathrm{AMO}_E(G). \end{aligned} \tag{17}$$

Each row and column therefore contains exactly one queen, while each diagonal contains at most one queen. The implementation retains column ALO clauses even though they follow from row exactly-one and column AMO constraints. Keeping the same clauses in all configurations ensures a consistent comparison between AMO encodings. No additional manual symmetry-breaking constraints are added.

The encoders construct CNF clauses directly in Python. PySAT is used to access Glucose3 and CaDiCaL 1.9.5, rather than to generate cardinality encodings through `CardEnc` [2]. Each of the five encodings is evaluated with both SAT solvers, giving ten SAT configurations.

After a satisfiable result, only positive literals corresponding to the first N^2 variables are used to recover the board. The decoder requires exactly one selected cell in each row and converts the result into the common representation $q_i = j$. Auxiliary variables are excluded from the decoded placement.

3.2. At-most-one encodings

Different CNF encodings of cardinality constraints can differ substantially in size and structure [3]. The following descriptions specify the variants implemented in this project.

For an AMO group of size m, let $a_E(m)$ be the number of new auxiliary variables and $c_E(m)$ the number of generated clauses. These counts exclude any ALO clause. For every encoding,

$$a_E(m) = c_E(m) = 0 \qquad \text{for } m \in \{0, 1\}. \tag{18}$$

3.2.1. Pairwise encoding

Pairwise, also named Binomial in the implementation, forbids every pair of input variables from being true simultaneously:

$$\mathrm{AMO}_{\mathrm{PW}}(Y) = \bigwedge_{1 \le i < j \le m} (\neg y_i \vee \neg y_j). \tag{19}$$

It introduces no auxiliary variables:

$$a_{\mathrm{PW}}(m) = 0, \qquad c_{\mathrm{PW}}(m) = \binom{m}{2}. \tag{20}$$

All clauses are binary, but their number grows quadratically with the group size.

3.2.2. Binary encoding

For $m \geq 2$, Binary encoding introduces $b = \lceil \log_2 m \rceil$ auxiliary bits $z_0, \ldots, z_{b-1}$. Input y_i is assigned the binary code of $i - 1$. Let

$$\ell_{it} = \begin{cases} z_t, & \text{if bit } t \text{ of } i-1 \text{ is 1,} \\ \neg z_t, & \text{otherwise.} \end{cases} \tag{21}$$

The encoder generates

$$\mathrm{AMO}_{\mathrm{BIN}}(Y) = \bigwedge_{i=1}^{m} \bigwedge_{t=0}^{b-1} (\neg y_i \vee \ell_{it}). \tag{22}$$

If two distinct inputs are true, their codes disagree on at least one bit, producing a contradiction. If no input is true, the auxiliary bits remain unrestricted; this is permitted because the constraint is AMO rather than ALO.

The exact counts are

$$a_{\mathrm{BIN}}(m) = \lceil \log_2 m \rceil, \qquad c_{\mathrm{BIN}}(m) = m \lceil \log_2 m \rceil. \tag{23}$$

3.2.3. Commander encoding

The implemented Commander encoding partitions the input list into consecutive groups $G_1, \ldots, G_g$ of at most three variables, where $g = \lceil m/3 \rceil$. For $m \leq 3$, it uses Pairwise directly.

For $m > 3$, each group G_j receives a commander variable u_j. The encoder imposes Pairwise AMO within each group and adds the linking clauses

$$(\neg y \vee u_j), \qquad \forall y \in G_j, \tag{24}$$

$$\left(\neg u_j \vee \bigvee_{y \in G_j} y\right), \qquad j = 1, \ldots, g. \tag{25}$$

Together, these clauses express $u_j \leftrightarrow \bigvee_{y \in G_j} y$. Commander encoding is then applied recursively to $(u_1, \ldots, u_g)$.

Thus, two true inputs in the same group violate its local Pairwise constraint, while true inputs in different groups would require two commanders to be true.

Writing $t_j = |G_j|$, the exact recurrences for $m > 3$ are

$$a_{\mathrm{CMD}}(m) = g + a_{\mathrm{CMD}}(g), \tag{26}$$

$$c_{\mathrm{CMD}}(m) = m + g + \sum_{j=1}^{g} \binom{t_j}{2} + c_{\mathrm{CMD}}(g). \tag{27}$$

The base cases are $a_{\mathrm{CMD}}(m) = 0$ and $c_{\mathrm{CMD}}(m) = \binom{m}{2}$ for $m \leq 3$. The term g in Equation (27) accounts for the reverse linking clauses; it must be included when counting this particular implementation.

3.2.4. Recursive Product encoding

For groups of size $m \leq 4$, the Product implementation uses Pairwise encoding. For $m > 4$, it defines

$$p = \lceil \sqrt{m} \rceil, \qquad r = \left\lceil \frac{m}{p} \right\rceil, \tag{28}$$

and assigns the inputs to distinct cells of a conceptual $p \times r$ grid in row-major order.

Introduce selector variables $u_1, \ldots, u_p$ for the grid rows and $v_1, \ldots, v_r$ for its columns. If input y_i occupies grid cell (a_i, b_i), add

$$(\neg y_i \vee u_{a_i}) \wedge (\neg y_i \vee v_{b_i}). \tag{29}$$

The same Product encoder is applied recursively to both selector groups.

Two distinct inputs occupy different grid rows or different grid columns. If both inputs are true, at least one selector group would contain two true variables, violating its AMO constraint. No reverse implications from selectors to inputs are generated.

For $m > 4$, the exact counts satisfy

$$a_{\mathrm{PROD}}(m) = p + r + a_{\mathrm{PROD}}(p) + a_{\mathrm{PROD}}(r), \tag{30}$$

$$c_{\mathrm{PROD}}(m) = 2m + c_{\mathrm{PROD}}(p) + c_{\mathrm{PROD}}(r). \tag{31}$$

For $m \leq 4$, $a_{\mathrm{PROD}}(m) = 0$ and $c_{\mathrm{PROD}}(m) = \binom{m}{2}$. These recurrences include all recursive levels and the Pairwise base cases.

3.2.5. Sequential Counter encoding

The implementation uses the AMO specialization of the sequential-counter construction [4]. For $m \geq 2$, it introduces auxiliary variables $s_1, \ldots, s_{m-1}$ to propagate information about true inputs along the sequence.

The two endpoint clauses are

$$(\neg y_1 \vee s_1) \wedge (\neg y_m \vee \neg s_{m-1}). \tag{32}$$

For each $i = 2, \ldots, m-1$, the encoder adds

$$(\neg y_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg y_i \vee \neg s_{i-1}). \tag{33}$$

A true input forces the corresponding signal and subsequent signals to be true, preventing any later input from also being true. The clauses need not uniquely determine all auxiliary values when the inputs are false. The exact counts are

$$a_{\mathrm{SEQ}}(m) = m - 1, \qquad c_{\mathrm{SEQ}}(m) = 3m - 4. \tag{34}$$

3.3. Variable and clause analysis

The following analysis counts the generated CNF before solver preprocessing. Total variables include both the N^2 board variables and all allocated auxiliary variables. Clause counts include the row and column ALO clauses and all clauses generated by the selected AMO encoding, without deduplication.

For $N \geq 2$, there are $2N$ row and column groups of length N. Across both diagonal directions, there are two groups of length N and four groups of each length $2, \ldots, N-1$. Thus, the formulation contains $6N - 6$ nontrivial AMO groups and $2N$ ALO clauses.

Using the per-group auxiliary-variable count $a_E(m)$ and clause count $c_E(m)$ defined above, the total counts are

$$V_E(N) = N^2 + (2N+2)a_E(N) + 4\sum_{\ell=2}^{N-1} a_E(\ell), \tag{35}$$

$$C_E(N) = 2N + (2N+2)c_E(N) + 4\sum_{\ell=2}^{N-1} c_E(\ell). \tag{36}$$

An empty sum is interpreted as zero. For $N = 1$, the implementation produces one board variable and two identical unit ALO clauses, with no AMO clauses.

For Pairwise, substitution gives

$$V_{\mathrm{PW}}(N) = N^2, \qquad C_{\mathrm{PW}}(N) = \frac{5N^3 - 6N^2 + 7N}{3}. \tag{37}$$

For Sequential Counter and $N \geq 2$, the closed forms are

$$V_{\mathrm{SEQ}}(N) = 5N^2 - 6N + 2, \tag{38}$$

$$C_{\mathrm{SEQ}}(N) = 12N^2 - 22N + 12. \tag{39}$$

The Binary counts follow by substituting $a_{\mathrm{BIN}}(m) = \lceil \log_2 m \rceil$ and $c_{\mathrm{BIN}}(m) = m\lceil \log_2 m \rceil$. Commander and Product are evaluated using their exact recurrences, including the implemented group sizes and Pairwise base cases.

Tables 1 and 2 report these counts for all 18 board sizes used in the benchmark. The values agree with the project's exported CNF-size data. They are structural counts calculated from the encoding formulas; they do not imply that every corresponding CNF was fully constructed or successfully solved within the experimental resource limits. In both tables, Product denotes recursive Product and Sequential denotes Sequential Counter.

Table 1
Total numbers of SAT variables, including board and auxiliary variables, before solver preprocessing.

N	Pairwise	Binary	Commander	Product	Sequential
4	16	48	36	16	58
8	64	174	154	236	274
9	81	229	189	289	353
16	256	572	744	820	1,186
25	625	1,241	1,813	2,533	2,977
27	729	1,405	2,061	2,937	3,485
36	1,296	2,328	3,876	4,732	6,266
49	2,401	3,901	7,265	8,097	11,713
64	4,096	6,136	12,442	12,476	20,098
81	6,561	9,441	19,449	18,221	32,321
100	10,000	13,678	30,536	26,104	49,402
125	15,625	20,353	47,257	37,749	77,377
216	46,656	55,988	140,030	94,672	231,986
343	117,649	134,109	355,969	220,149	586,189
512	262,144	287,730	789,618	459,472	1,307,650
729	531,441	571,069	1,595,061	859,721	2,652,833
1000	1,000,000	1,055,888	3,013,684	1,514,732	4,994,002
1331	1,771,561	1,851,197	5,326,901	2,536,565	8,849,821

At the board level, Pairwise requires $\Theta(N^3)$ clauses, Binary requires $\Theta(N^2 \log N)$, and Commander, recursive Product, and Sequential Counter each require $\Theta(N^2)$. All five formulations have $\Theta(N^2)$ total variables, although their auxiliary-variable counts differ. Binary uses $\Theta(N \log N)$ auxiliary variables, recursive Product uses $\Theta(N^{3/2})$, and Commander and Sequential Counter use $\Theta(N^2)$.

The tables illustrate the trade-off between variables and clauses. Pairwise introduces no auxiliary variables and therefore has the smallest variable count, but its clause count grows rapidly. At $N = 1331$, it requires 3,926,372,802 clauses, compared with 15,533,774 for recursive Product.

Binary keeps the auxiliary-variable count relatively small, but produces more clauses than Commander, Product, and Sequential Counter at the larger tested sizes. Recursive Product has the smallest clause count among the five encodings at every tested size from $N = 49$ onward. At $N = 4$, it coincides with Pairwise because all AMO groups fall within its Pairwise base case.

These structural differences do not establish a runtime ranking. Clause structure, propagation, preprocessing, and search behavior also affect performance. Similarly, observed memory includes Python objects, native solver structures, and process overhead, rather than only the generated variable and clause counts. The experimental evaluation therefore considers CNF size together with runtime, memory consumption, and solution success.

Table 2
Total numbers of generated CNF clauses, including ALO and AMO clauses, before solver preprocessing.

N	Pairwise	Binary	Commander	Product	Sequential
4	84	120	124	84	116
8	744	728	634	652	604
9	1,074	1,114	822	826	786
16	6,352	3,928	3,176	2,804	2,732
25	24,850	11,806	8,018	7,638	6,962
27	31,410	13,890	9,294	8,930	8,166
36	75,252	28,320	17,276	15,292	14,772
49	191,394	54,866	32,526	27,742	27,746
64	428,864	95,576	55,990	45,620	47,756
81	872,802	172,694	89,322	70,710	76,962
100	1,646,900	269,024	138,628	106,292	117,812
125	3,224,250	426,574	216,062	161,882	184,762
216	16,703,352	1,449,224	647,438	458,024	555,132
343	67,021,514	4,060,266	1,644,150	1,121,094	1,404,254
512	223,173,120	9,262,424	3,662,390	2,439,384	3,134,476
729	644,639,634	20,558,002	7,421,982	4,824,850	6,361,266
1000	1,664,669,000	39,300,904	13,999,546	8,907,744	11,978,012
1331	3,926,372,802	75,151,050	24,787,642	15,533,774	21,229,462

3.4. Correctness arguments and validation

3.4.1. Correctness of the encoding construction

Let $A_E(Y, Z)$ denote the CNF generated for one AMO group, where Y contains the input variables and Z contains its auxiliary variables. The required property is

$$\mathrm{AMO}(Y) \quad \Longleftrightarrow \quad \exists Z \; A_E(Y, Z). \tag{40}$$

The existential quantifier is important: the encodings preserve valid assignments to the inputs without requiring a unique assignment to the auxiliary variables.

The constructions reject assignments with two or more true inputs for the reasons described above: Pairwise forbids every true pair, Binary assigns incompatible codes, Commander separates conflicts within and between groups, Product separates inputs by grid coordinates, and Sequential Counter propagates a signal that excludes later true inputs.

Conversely, every assignment with at most one true input can be extended to satisfy the generated clauses. Binary bits can encode the selected input, if one exists. Commander variables can indicate whether their groups contain that input. Product selectors can identify its grid row and column. Sequential signals can be assigned the prefix disjunctions $s_i = \bigvee_{j=1}^{i} y_j$. The recursive constructions apply the same argument at each level, terminating at valid Pairwise base cases.

Because different AMO groups receive disjoint auxiliary variables, these extensions can be combined. Together with the row and column ALO clauses, this establishes that satisfying assignments of $F_E(N)$ project exactly onto valid N-Queens placements.

3.4.2. Automated encoder tests

The unit-test suite checks all five AMO encoders for

$$m \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 10\}. \tag{41}$$

For every assignment to the m input variables, the test fixes those inputs using solver assumptions and checks whether the remaining formula is satisfiable. The expected result is satisfiable if and only if at most one input is true. Auxiliary variables remain free during these checks, matching Equation (40).

Additional tests exercise recursive Product on groups of sizes 17, 25, and 100. These tests use nonconsecutive input identifiers and reserve an identifier range before allocating auxiliaries. They check the all-false assignment, every assignment with exactly one true input, and the inconsistency of every pair of true inputs.

The suite also tests complete N-Queens formulas for all five encodings with both SAT solvers at

$$N \in \{1, 2, 3, 4, 5, 6, 8\}. \tag{42}$$

The expected outcomes are UNSAT for $N = 2, 3$ and SAT for the remaining sizes. Returned placements are decoded and validated independently. These checks comprise 50 AMO test cases, 70 complete SAT model test cases, and three additional recursive Product test cases, for a total of 123 parameterized tests.

3.4.3. Validation across solver configurations

A separate correctness runner checks all 14 configurations, including the CP and MIP baselines, on the same seven board sizes. The saved complete run records 98 passed checks, zero failures, and zero skipped configurations. These correctness runs are separate from the performance benchmark.

For a reported solution on an $N \times N$ board, validation requires a vector of length N, integer column values in $\{0, \dots, N-1\}$, and

$$\begin{aligned} |\{q_i : i \in \mathcal{I}\}| &= N, \\ |\{i - q_i : i \in \mathcal{I}\}| &= N, \\ |\{i + q_i : i \in \mathcal{I}\}| &= N. \end{aligned} \tag{43}$$

The validator examines the decoded placement directly; it does not rely on the clauses or constraints used to construct the solver model.

The finite tests provide implementation checks in addition to the construction-level correctness argument. They are not an exhaustive test of every board size. A time limit, memory limit, or abnormal termination is recorded as an unsuccessful computational outcome and is never interpreted as proof of infeasibility.

4. Experiments and Results

This section evaluates the implemented configurations in terms of solution success, total wall time, and observed peak memory. The main benchmark contains 756 runs. Supplementary diagnostic reruns are reported separately and do not replace any original observation.

4.1. Experimental setup

The experiments were conducted on a Windows 11 machine with an Intel x86-64 processor, six physical cores, twelve logical processors, and approximately 15.73 GiB of physical memory reported by the operating system. Table 3 summarizes the software environment recorded for the benchmark.

The tested board sizes are

$$\begin{aligned} \mathcal{N} = \{&4, 8, 9, 16, 25, 27, 36, 49, 64, \\ &81, 100, 125, 216, 343, 512, 729, 1000, 1331\}. \end{aligned} \tag{44}$$

The benchmark combines five SAT encodings with two SAT solvers and includes four additional CP/MIP configurations. With three repetitions for each configuration and size, the planned number of runs is

$$14 \times 18 \times 3 = 756. \tag{45}$$

Each run executes in a separate worker process under an external wall-time budget of 100 seconds and a sampled process-tree RSS threshold of 8192 MiB, equivalent to 8 GiB. The monitor samples memory approximately every 50 milliseconds. Only one benchmark run is active at a time. The configured execution order is repetition, board size, and then method order; it is not randomized.

The resource budgets apply to the complete worker execution, including model construction and solver initialization. Consequently, a run may exhaust its resources before reaching the solver's search routine. The memory threshold

Table 3
Recorded experimental environment and execution settings.

Component	Configuration
Operating system	Windows 11, build 26200
CPU resources	6 physical cores, 12 logical processors
Physical memory	Approximately 15.73 GiB
Python	3.12.10
PySAT package	`python-sat` 1.9.dev15
SAT backends	Glucose3 and CaDiCaL 1.9.5
OR-Tools package	9.15.6755
IBM optimization suite	CPLEX Optimization Studio 22.2
CPLEX Python package	22.2.0.1
DOcplex package	2.32.264
Gurobi Python package	13.0.3
Memory monitoring	`psutil` 7.2.2
Concurrent benchmark runs	One
CP/MIP workers or threads	One per solver
Solver seeds	Defaults; no explicit seed schedule

is enforced by monitoring and terminating the process tree, rather than by an operating-system hard allocation limit. A recorded peak can therefore slightly exceed the configured threshold.

After the first 239 recorded runs, execution was interrupted under Windows memory pressure. The paging-file configuration was changed and process cleanup in the resource monitor was revised before the remaining 517 runs were completed. The stored source hashes show no changes to the encoders or solver wrappers between these phases. The implications of this interruption are discussed in Section 4.7.

4.2. Measurements and aggregation

A run is counted as successful only when it returns a placement that passes independent validation. For consistent reporting across SAT, CP, and MIP backends, such a result is recorded as `SAT`. Resource exhaustion and abnormal termination are retained as distinct outcomes.

The primary runtime measure is `total_wall_s`, measured by the external runner. It includes process startup, imports, model construction, solver execution, result handling, and cleanup. This measure represents the observed cost of the complete implementation rather than solver search alone.

The wrappers additionally record `build_s` and `solve_s`. For SAT, construction time includes both generating the Python clause list and loading it into the native solver. Solver-call time surrounds the call to `solve()`. For CP and MIP, the timed API call may also include model transfer, engine startup, or preprocessing. These components are therefore interpreted within each wrapper, rather than assumed to isolate identical operations across all solver families.

Memory usage is measured as the highest sampled sum of RSS over the worker and its descendant processes. It includes implementation overhead in addition to the solver's internal data structures.

Runtime and memory summaries use the median and interquartile range (IQR) of successful runs for each configuration and size. The plotted shaded regions extend from the first quartile to the third quartile; they are descriptive ranges, not confidence intervals. Failed runs are not assigned artificial successful runtimes, and groups without successful runs have no runtime or memory summary. Consequently, performance plots must be read together with the success counts.

4.3. Overall outcomes

All 756 planned combinations of configuration, size, and repetition are present, with no duplicate run keys. The benchmark records 612 valid solutions, 101 timeouts, 42 memory-limit or allocation-failure outcomes, and one abnormal termination. The overall success rate is therefore

$$\frac{612}{756} \times 100\% \approx 80.95\%. \tag{46}$$

No run reports UNSAT, which is consistent with all benchmark sizes being at least four.

Table 4
Outcomes across 54 runs per configuration. T denotes `TIMEOUT`, M denotes `MEMORY_LIMIT`, and E denotes `ERROR`.

Solver	Encoding/model	SAT	T	M	E	Success (%)
Glucose3	Pairwise	39	0	15	0	72.2
Glucose3	Binary	45	3	6	0	83.3
Glucose3	Commander	54	0	0	0	100.0
Glucose3	Product	21	33	0	0	38.9
Glucose3	Sequential	33	21	0	0	61.1
CaDiCaL	Pairwise	39	0	15	0	72.2
CaDiCaL	Binary	48	0	5	1	88.9
CaDiCaL	Commander	51	2	1	0	94.4
CaDiCaL	Product	48	6	0	0	88.9
CaDiCaL	Sequential	42	12	0	0	77.8
OR-Tools	CP-SAT	39	15	0	0	72.2
IBM	CP Optimizer	54	0	0	0	100.0
IBM	CPLEX MIP	48	6	0	0	88.9
Gurobi	MIP	51	3	0	0	94.4
Total		612	101	42	1	80.95

Table 4 gives the outcome counts for each configuration. Every configuration has 54 planned runs.

IBM CP Optimizer and Glucose3 with Commander are the only configurations that solve all 54 planned runs. CaDiCaL with Commander and Gurobi MIP each solve 51 runs, while several other configurations encounter resource limits at larger sizes.

Figure 2 shows success by board size. In this dataset, each configuration–size group has either three successful repetitions or none. The repetitions therefore agree on whether a valid solution is obtained, although the failure category can differ within an unsuccessful group.

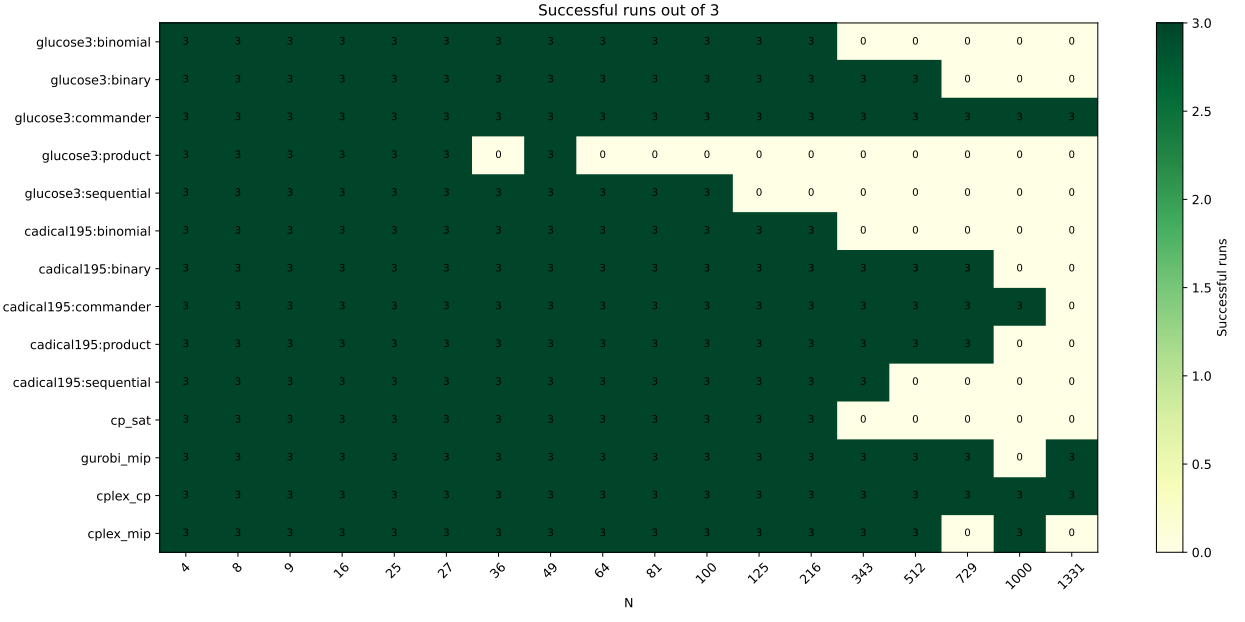


Figure 2: Successful runs out of three repetitions for each configuration and board size. The labels `binomial`, `sequential`, and `cplex_cp` denote Pairwise, Sequential Counter, and IBM CP Optimizer, respectively.

4.4. Runtime and scalability

Figure 3 presents median total wall time and the first-to-third-quartile range for successful runs. Both axes use logarithmic scales. Missing points indicate that no successful runtime is available for that configuration and size.

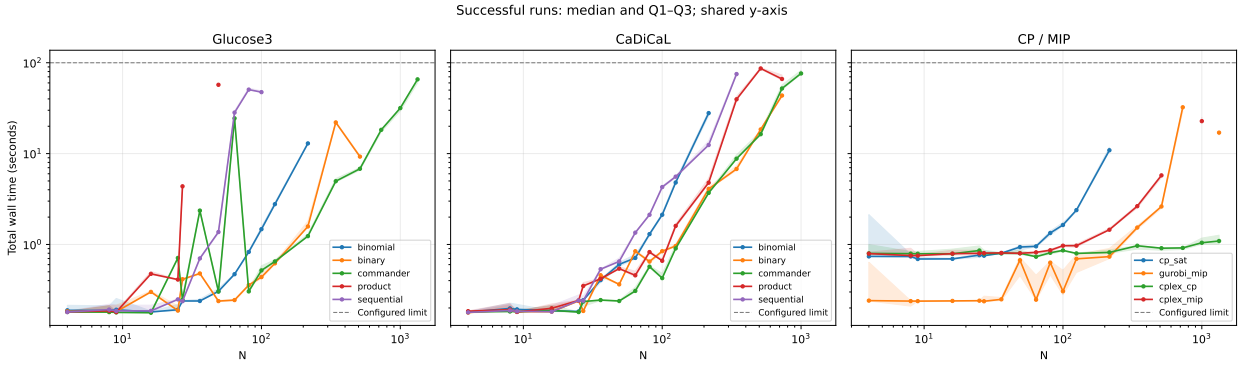


Figure 3: Total wall time for successful runs. Points show medians and shaded regions show Q1–Q3. The dashed line marks the configured 100-second budget. Unsuccessful groups have no plotted runtime.

4.4.1. Comparison of SAT configurations

The results show that encoding performance depends strongly on the selected solver. The clearest example is recursive Product: it solves 21 of 54 runs with Glucose3 but 48 of 54 with CaDiCaL. At $N = 49$, the median total wall times are 57.202 seconds with Glucose3 and 0.542 seconds with CaDiCaL. At $N = 64$, all three Glucose3 Product runs time out, whereas CaDiCaL Product succeeds in all three repetitions with a median total wall time of 0.457 seconds.

Glucose3 Commander succeeds across the full tested range. Its median total wall time is 31.714 seconds at $N = 1000$ and 65.643 seconds at $N = 1331$. CaDiCaL Commander also succeeds at $N = 1000$, with a median of 76.232 seconds, but its three runs at $N = 1331$ result in two timeouts and one memory-limit outcome.

Sequential Counter reaches different practical limits with the two SAT backends. Glucose3 solves all tested sizes up to $N = 100$ and times out at every larger tested size. CaDiCaL succeeds through $N = 343$ and times out at all tested sizes from $N = 512$ onward. Thus, a linear clause bound for individual AMO groups does not guarantee that the resulting N-Queens formulas will be solved within the selected budget.

Observed runtime is not monotonic in board size. For example, Glucose3 Commander takes a median of 24.456 seconds at $N = 64$ but only 0.518 seconds at $N = 100$. Such differences demonstrate why runtime cannot be predicted from board size or generated clause count alone. The present measurements do not isolate the internal search mechanisms responsible for these variations.

4.4.2. Comparison with CP and MIP baselines

IBM CP Optimizer solves every tested instance in all three repetitions. At the largest size, its median total wall time is 1.092 seconds. This is substantially lower than the corresponding times of the other configurations that successfully solve $N = 1331$ under the same external budgets.

Table 5 summarizes those successful configurations. All three have three validated runs at $N = 1331$; the remaining configurations have none.

Table 5
Successful configurations at $N = 1331$. Runtime and memory values summarize three successful runs. RSS is measured in MiB.

Configuration	Solved	Median time (s)	IQR (s)	Median RSS
IBM CP Optimizer	3/3	1.092	0.258	133.93
Gurobi MIP	3/3	17.021	1.257	1178.81
Glucose3 + Commander	3/3	65.643	5.969	5987.35

The two CP-based configurations differ substantially despite receiving the same high-level integer-variable formulation. OR-Tools CP-SAT solves all tested sizes through $N = 216$ but times out at every tested size from $N = 343$ onward. Consequently, the compact model with three global distinctness constraints does not by itself explain CP Optimizer's performance; backend processing and search also matter.

The MIP results are likewise nonmonotonic. Gurobi times out in all three runs at $N = 1000$ but succeeds in all three at $N = 1331$, with a median total wall time of 17.021 seconds. CPLEX MIP times out at $N = 729$, succeeds at $N = 1000$ with a median of 22.768 seconds, and times out again at $N = 1331$. A largest successful board size alone would therefore give an incomplete account of either solver's behavior.

4.4.3. Construction time and execution overhead

The recorded time components show why total wall time is needed for comparison. For Glucose3 Commander at $N = 1331$, the median construction time is 31.648 seconds and the median solver-call time is 29.841 seconds. Construction, including loading the clauses into the solver, is therefore a substantial part of the observed cost.

For Gurobi at the same size, the median construction time is 12.454 seconds, while the median optimization-call time is 0.747 seconds. Its median total wall time is nevertheless 17.021 seconds. For IBM CP Optimizer, the median solver-call time is 0.189 seconds, compared with 1.092 seconds for the complete worker execution.

These values are medians calculated separately for each measurement. They need not sum exactly, and total wall time also includes operations outside the construction and solver-call timers. Reporting only the solver-call duration would omit a substantial portion of the implementation

4.4.4. Detailed runtime comparison

Table 6 provides a numerical comparison of four selected SAT configurations and the four CP/MIP baselines across all tested board sizes. The selected SAT configurations are Commander with Glucose3, and Commander, Binary, and recursive Product with CaDiCaL. Results for all 14 configurations remain available in the overall outcome table and runtime plots.

Each numerical entry is the median total wall time of three successful repetitions, measured in seconds. Total wall time includes worker startup, imports, construction, solver execution, result handling, and cleanup. When all three repetitions fail, the cell reports their failure categories instead of a numerical runtime. Bold entries identify the lowest unrounded median among the eight configurations displayed in that row; they do not imply statistically significant differences.

The detailed results reinforce the nonmonotonic behavior observed in the runtime plots. For example, Glucose3 Commander is considerably slower at $N = 64$ than at $N = 100$, while Gurobi fails at $N = 1000$ but succeeds at $N = 1331$. IBM CP Optimizer has the lowest median among the displayed configurations at every tested size from $N = 343$ onward. These observations apply to the recorded implementations and execution conditions; differences on small instances can also reflect startup overhead and run-to-run variation..

Table 6
Median total wall time in seconds for selected SAT, CP, and MIP configurations. CG: Commander with Glucose3; CC: Commander with CaDiCaL; BC: Binary with CaDiCaL; PC: recursive Product with CaDiCaL. CP Opt. denotes IBM CP Optimizer. Bold indicates the lowest median among the displayed configurations at each size.

N	CG	CC	BC	PC	CP-SAT	CP Opt.	Gurobi	CPLEX MIP
4	0.181	0.181	**0.178**	0.184	0.738	0.797	0.241	0.793
8	**0.180**	0.183	0.190	0.192	0.738	0.793	0.238	0.754
9	**0.179**	0.182	0.182	0.181	0.692	0.790	0.238	0.754
16	**0.177**	0.188	0.182	0.198	0.694	0.790	0.239	0.793
25	0.710	**0.180**	0.239	0.242	0.763	0.858	0.240	0.798
27	0.246	0.235	**0.185**	0.350	0.752	0.793	0.237	0.803
36	2.360	**0.244**	0.458	0.419	0.801	0.798	0.249	0.807
49	0.303	**0.238**	0.364	0.542	0.937	0.800	0.671	0.799
64	24.456	0.308	0.842	0.457	0.953	0.733	**0.247**	0.819
81	**0.304**	0.570	0.648	0.825	1.338	0.810	0.631	0.863
100	0.518	0.426	0.843	0.661	1.638	0.860	**0.305**	0.969
125	**0.650**	0.904	0.961	1.600	2.387	0.797	0.694	0.970
216	1.234	3.697	4.085	4.783	10.888	0.818	**0.735**	1.453
343	4.969	8.794	6.773	39.599	TO	**0.968**	1.535	2.639
512	6.810	16.351	18.452	86.519	TO	**0.910**	2.619	5.758
729	18.267	52.080	43.590	66.267	TO	**0.916**	32.349	TO
1000	31.714	76.232	ML/E	TO	TO	**1.044**	TO	22.768
1331	65.643	TO/ML	ML	TO	TO	**1.092**	17.021	TO
Solved/18	18	17	16	16	13	18	17	16

TO: all three repetitions timed out. ML: all three repetitions received the memory-limit classification. ML/E: two memory-limit outcomes and one error (BC at $N = 1000$). TO/ML: two timeouts and one memory-limit outcome (CC at $N = 1331$). A numerical entry summarizes three validated solutions. Solved/18 counts board sizes with three successful repetitions, rather than individual successful runs.

4.5. Memory consumption and model size

Figure 4 shows the median sampled peak RSS and Q1–Q3 range for successful runs. As with the runtime figure, unsuccessful groups are omitted from the numerical summaries and must be interpreted using Table 4 and Figure 2.

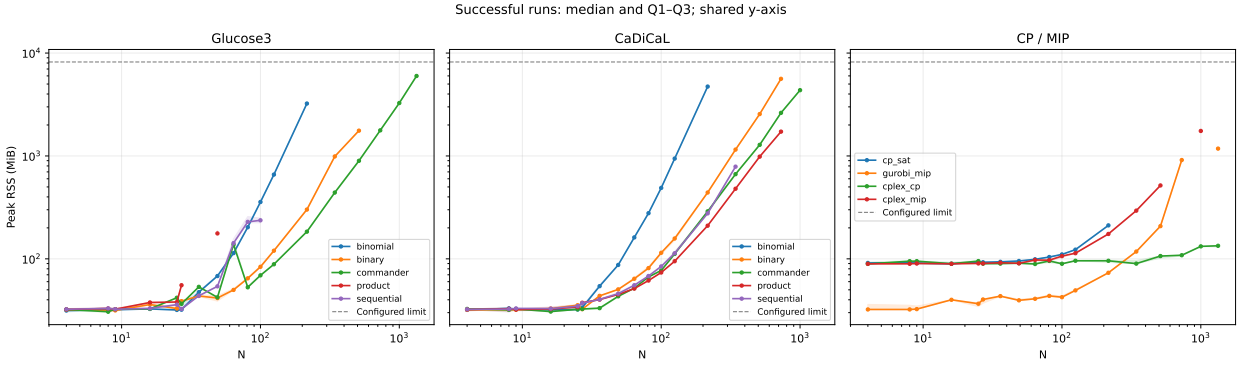


Figure 4: Sampled peak process-tree RSS for successful runs. Points show medians and shaded regions show Q1–Q3. The dashed line marks the configured 8192 MiB threshold. Memory-limit failures are not represented as successful memory measurements.

Pairwise exhibits the strongest growth in generated clause count among the implemented encodings, as established in Section 3.3. Both SAT backends solve the tested sizes through $N = 216$, but all Pairwise runs at $N = 343$ and

larger tested sizes receive the MEMORY_LIMIT classification. At $N = 216$, the median sampled peak RSS is already 3231.44 MiB for Glucose3 and 4725.21 MiB for CaDiCaL.

Binary introduces relatively few auxiliary variables, but its many clauses still create substantial memory demand. CaDiCaL Binary reaches a median peak RSS of 5618.84 MiB at $N = 729$. Neither SAT backend returns a solution with Binary at $N = 1000$ or $N = 1331$.

At $N = 1331$, the successful configurations also differ substantially in memory usage. IBM CP Optimizer has a median sampled peak RSS of 133.93 MiB, compared with 1178.81 MiB for Gurobi MIP and 5987.35 MiB for Glucose3 Commander.

These observations concern the complete implementations, not just their mathematical models. For SAT, the Python clause list remains allocated while the native solver constructs its own representation. The number of auxiliary variables alone is therefore an insufficient predictor of peak memory.

The MEMORY_LIMIT category includes both monitor termination and detected allocation failures. Four of the 42 records in this category have a sampled peak below 8192 MiB. Accordingly, this status must not be interpreted as evidence that every such run was observed crossing the RSS threshold.

4.6. Abnormal termination and supplementary diagnostics

One original run, CaDiCaL with Binary at $N = 1000$, terminates abnormally. It records a total wall time of 54.315 seconds, a sampled peak RSS of 8133.86 MiB, and Windows exit code 0xc0000409. The original log does not establish the precise cause, so the run remains classified as ERROR.

Two supplementary diagnostic runs were performed with the same encoder and solver source and the same 100-second and 8192 MiB budgets. One used the current runner, and the other additionally recorded periodic Python stack traces. Both were terminated by the monitor after crossing the sampled RSS threshold, without returning a solution.

Table 7
Original abnormal run and supplementary diagnostics for CaDiCaL Binary at $N = 1000$. Diagnostic runs are excluded from the main benchmark statistics.

Run	Status	Total time (s)	Peak RSS (MiB)
Original benchmark	ERROR	54.315	8133.86
Diagnostic rerun	MEMORY_LIMIT	54.762	8196.35
Stack-trace diagnostic	MEMORY_LIMIT	34.357	8199.81

In the stack-trace diagnostic, the sample near 15 seconds shows Binary clause generation. The sample near 30 seconds shows clause loading during CaDiCaL initialization. The last recorded stack is therefore in construction rather than in the explicit solve() call. Because stacks were sampled periodically, it does not identify the exact operation executing when termination occurred.

The expected Binary formula at this size contains 39,300,904 clauses and 1,055,888 variables. Together with the observed initialization stack, this makes construction-related memory pressure a relevant explanation for the diagnostic outcomes. However, the original abnormal exit was not reproduced, and its cause remains unconfirmed.

The two additional memory-limit outcomes are reported only as diagnostic evidence. They are not used to replace the original error, increase the repetition count, or recompute the main success rates, medians, or IQRs.

4.7. Interpretation and limitations

Within the tested implementations and budgets, IBM CP Optimizer provides complete success across the selected sizes and low runtime and memory use at the largest size. Glucose3 Commander also achieves complete success and is the only SAT configuration to solve all planned runs. However, the Product results demonstrate that an encoding's observed performance can change substantially with the SAT backend. There is no solver-independent ranking of all five encodings supported by these measurements.

Several limitations qualify these findings.

First, each configuration–size group has only three repetitions, and no explicit schedule of different solver seeds is used. The repetitions characterize observed variation under the recorded defaults, rather than a broad sample of independent search strategies. The IQR bands should not be interpreted as statistical confidence intervals.

Second, the paging-file and process-cleanup changes divide the benchmark into phases of 239 and 517 runs. Although encoder and solver-wrapper hashes remain unchanged, the execution environment is not fully uniform. The fixed, nonrandomized run order also leaves possible order and background-load effects uncontrolled.

Third, RSS is sampled approximately every 50 milliseconds. Short-lived peaks may be missed, while summing RSS across processes can count shared memory more than once. The configured threshold is a monitored termination rule, not a hard cap on all memory allocations or virtual memory.

Fourth, runtime and memory summaries are conditional on success. A configuration with few successful runs can appear fast on the limited cases it solves. For this reason, successful-run medians are not used alone to establish an overall ranking.

Fifth, the measurements include Python model construction, solver interfaces, initialization, and cleanup. They compare complete implementations on one machine, rather than isolating the performance of the underlying search algorithms.

Finally, the benchmark addresses ordinary N-Queens feasibility on a selected set of board sizes. It does not evaluate solution enumeration, N-Queens Completion, or all intermediate values of N. The observed nonmonotonic behavior also prevents interpreting a failed size as a universal upper limit for a solver.

5. Conclusion

This study compared five manually implemented SAT encodings using Glucose3 and CaDiCaL with four CP/MIP configurations for finding one feasible N-Queens placement. The evaluation covered 18 board sizes from $N = 4$ to $N = 1331$, with three repetitions per configuration and size. Under an external wall-time budget of 100 seconds and a sampled process-tree RSS threshold of 8 GiB, the 756 planned runs produced 612 independently validated solutions, 101 timeouts, 42 memory-limit or allocation-failure outcomes, and one abnormal termination.

IBM CP Optimizer and Glucose3 with Commander were the only configurations to solve all 54 planned runs. At $N = 1331$, CP Optimizer achieved a median total wall time of 1.092 seconds and a median sampled peak RSS of 133.93 MiB, compared with 65.643 seconds and 5987.35 MiB for Glucose3 Commander. These observations identify CP Optimizer as particularly effective at the largest tested size and Commander as the most successful SAT encoding when paired with Glucose3 in this experiment.

The results also show that neither CNF size nor encoding choice alone determines practical performance. Recursive Product solved 21 of 54 runs with Glucose3 but 48 of 54 with CaDiCaL, demonstrating a substantial dependence on the solver backend. Pairwise avoided auxiliary variables but encountered memory limitations as its clause count increased. Several configurations exhibited nonmonotonic behavior across board sizes, so a larger board was not necessarily harder to solve within the selected budgets. Model construction and initialization also contributed substantially to total execution time, supporting the use of complete-process measurements alongside solver-call timing.

These conclusions are limited to the tested implementations, machine, board sizes, and resource budgets. Only three repetitions were performed with default solver seeds, and runtime and memory summaries were calculated from successful runs. The paging-file and process-cleanup changes after the first 239 runs introduced an additional source of environmental variation. Sampled RSS measurements may miss short-lived peaks and do not constitute a hard operating-system memory limit. Furthermore, the original CaDiCaL Binary abnormal termination remains unexplained: supplementary diagnostics encountered memory limits but did not reproduce that exit. The findings therefore support an empirical comparison of the recorded configurations rather than a universal ranking of SAT, CP, and MIP methods.

Future work should first repeat the benchmark under a fixed execution environment, with more repetitions, an explicit seed policy, and randomized configuration order. Additional hardware and separately defined resource budgets would help assess how well the observed behavior generalizes. Performance analysis could also include failure-aware summaries alongside successful-run medians.

A second direction is to reduce SAT construction overhead. Generating clauses incrementally and releasing temporary Python structures may reduce the overlap between the Python CNF representation and the native solver representation. Such changes should be evaluated as separate implementation variants, since they can affect both memory consumption and construction time. Further experiments could vary Commander group sizes, Product recursion thresholds, and symmetry-breaking constraints while checking correctness for each variant.

Finally, extending the study to N-Queens Completion or solution enumeration would test the methods on different computational tasks. These extensions should be reported separately from ordinary feasibility. Overall, the present results emphasize that reliable solver comparisons require attention to encoding structure, solver behavior, implementation overhead, and unsuccessful outcomes, together with independent validation of returned solutions.

Data Availability

The source code, experiment configurations, recorded benchmark results, correctness-check outputs, and analysis notebook are available in the project repository: https://github.com/nvchien3406/nqueens-assignment.

The main benchmark data are stored in `results/experiments/20261008T102037Z_10c488e7/`. This directory includes individual run records, consolidated CSV and JSONL results, environment metadata, and aggregated statistics. Tables and figures exported by the analysis notebook are provided in its `notebook_analysis/` subdirectory. Supplementary diagnostic reruns are identified separately and are excluded from the main benchmark statistics.

The repository README provides setup and execution instructions. Reproducing experiments with IBM CP Optimizer, IBM CPLEX, and Gurobi requires the corresponding solver installations and appropriate licenses; license credentials are not included in the repository.

References

[1] I. P. Gent, C. Jefferson, P. Nightingale, Complexity of n-Queens Completion, Journal of Artificial Intelligence Research 59 (2017) 815–848.

[2] A. Ignatiev, A. Morgado, J. Marques-Silva, PySAT: A Python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, Springer, 2018, pp. 428–437. doi:10.1007/978-3-319-94144-8_26.

[3] A. M. Frisch, P. A. Giannaros, SAT encodings of the at-most-*k* constraint: Some old, some new, some fast, some slow, in: Proceedings of the Ninth International Workshop on Constraint Modelling and Reformulation (ModRef 2010), 2010.

[4] C. Sinz, Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, volume 3709 of *Lecture Notes in Computer Science*, Springer, 2005, pp. 827–831. doi:10.1007/11564751_73.